BÁO CÁO BÀI TẬP LỚN

Phân tích so sánh thuật toán sắp xếp theo kích thước dữ liệu

Comparative Analysis of Sorting Algorithms for Different Data Sizes

Học phần Foundation Project

Trường: .....
Khoa / lớp: .....
Giảng viên: .....
Nhóm: .....

Thành viên	Họ và tên	Mã sinh viên
TV1	.....	.....
TV2	.....	.....
TV3	.....	.....
TV4	.....	.....

Kế hoạch học phần: 14/09/2026 đến 22/11/2026
Bộ số liệu trong báo cáo: phép đo thực hiện ngày 29/09/2026
Mã nguồn đo nhận diện bằng SHA256 trong metadata.json

Sản phẩm gồm chương trình CLI chạy độc lập, dữ liệu E1–E4, biểu đồ, bộ kiểm thử và tài liệu tái tạo. Các thông tin hành chính trên bìa do nhóm điền trước khi nộp.

Tóm tắt

Báo cáo xây dựng và đánh giá Insertion Sort, Merge Sort, Quick Sort dùng pivot cuối, Quick Sort dùng median-of-three và Heap Sort; Python sorted() được dùng làm baseline Timsort. Mục tiêu là liên hệ độ phức tạp với kết quả đo khi thay đổi kích thước và phân phối dữ liệu, đồng thời cung cấp chương trình và bằng chứng kiểm thử để tái tạo kết quả.

Ma trận có 216 cấu hình, gồm 180 cấu hình chính và 36 cấu hình kích thước nhỏ. Có 192 cấu hình đo thành công và 24 cấu hình bỏ qua theo giới hạn bậc hai, tạo 1728 quan sát thời gian. Các phép đo đếm và bộ nhớ được thực hiện riêng, không làm sai lệch thời gian bản thường.

Trên dữ liệu random, độ dốc log–log của Insertion là 2,111; Merge là 1,104; Quick pivot cuối là 1,142; Quick median-of-three là 1,114; Heap là 1,160. Các số này phù hợp xu hướng bậc hai và n log n trong miền khảo sát, nhưng không thay thế chứng minh tiệm cận. Timsort nhanh nhất trong các cấu hình đã đo do cả đặc tính thích nghi và cài đặt C.

Bộ kiểm thử đạt 137/137 test với 1.000 ví dụ Hypothesis và độ phủ dòng lõi 98.33%. Kiểm tra tái lập 6 cấu hình đều có sai khác median dưới 10%. Tuy nhiên 28/192 cấu hình có CV trên 10%, vì vậy gate ổn định thời gian của Plan chưa đạt toàn bộ. Kết quả được trình bày cùng giới hạn môi trường container dùng chung.

Mục lục

Chương 1 Giới thiệu và yêu cầu 3

Chương 2 Cơ sở lý thuyết và thiết kế 5

Chương 3 Thực nghiệm và kết quả 12

Kết luận 25

Tài liệu tham khảo 26

Phụ lục A Hướng dẫn chạy và demo 27

Phụ lục B Kiểm thử và nghiệm thu 28

Phụ lục C Phân công và hồ sơ bàn giao 29

Các tiêu đề trong báo cáo dùng Heading để điều hướng trong Word.

Chương 1 Giới thiệu và yêu cầu

1.1 Bài toán và động cơ

Sắp xếp xuất hiện trong tìm kiếm, thống kê, xử lý danh sách và tổ chức dữ liệu. Hai thuật toán cùng độ phức tạp O(n log n) có thể khác đáng kể về thời gian thực tế do số phép so sánh, ghi bộ nhớ, mức thích nghi với dữ liệu và chi phí ngôn ngữ. Ngược lại, một thuật toán bậc hai vẫn có thể phù hợp với mảng rất nhỏ.

Bài tập lớn triển khai một phòng thí nghiệm phần mềm thống nhất. Mỗi thuật toán trả một bản sao đã sắp xếp, cùng generator tạo dữ liệu, cùng quy trình đo và cùng cấu trúc CSV. Việc kiểm tra kết quả bằng sorted() trước khi ghi số liệu giúp tránh kết luận hiệu năng trên mã sai.

1.2 Mục tiêu và câu hỏi nghiên cứu

Bảng 1 Câu hỏi nghiên cứu

Mã	Câu hỏi	Cách trả lời
RQ1	Chi phí tăng thế nào khi n tăng?	E1, số đếm E3, doubling ratio và slope
RQ2	Phân phối ảnh hưởng ra sao?	E2 với 5 chế độ generator, heatmap tại n=4000
RQ3	Lựa chọn thuật toán trong trường hợp nào?	Kết hợp thời gian, ổn định, bộ nhớ và small-n

1.3 Phạm vi

Đầu vào là danh sách số nguyên trong RAM, đầu ra tăng dần. Các kernel hỗ trợ đối tượng có phép so sánh < nhất quán để kiểm thử tính ổn định. Không khảo sát sorting ngoài RAM, GPU, thuật toán phân tán, radix/counting sort, NaN hoặc kiểu dữ liệu trộn không so sánh được.

Kết quả thực nghiệm chỉ áp dụng cho cài đặt và môi trường được ghi nhận. Một seed chính giúp tái tạo chính xác dữ liệu nhưng chưa đủ để ước lượng phân bố hiệu năng trên mọi tập dữ liệu ngẫu nhiên. Python sorted() là baseline thư viện, không dùng để khẳng định thuật toán C ưu việt hơn thuần túy về mặt lý thuyết.

1.4 Tiêu chí hoàn thành

Các tiêu chí lấy từ Plan và phiếu nhiệm vụ bốn thành viên: 6 phiên bản, 5 phân phối, E1–E4, >=12 hình, >=60 test case, >=1000 ví dụ property-based, coverage lõi >=90%, CLI và lệnh tái tạo. Ma trận 100 task với vị trí tệp bàn giao nằm trong docs/PLAN_TRACEABILITY.csv. Những hoạt động cần sự tham gia của nhóm có trạng thái xác nhận riêng.

1.5 Yêu cầu và phân công

Bảng 2 Yêu cầu chức năng

ID	Yêu cầu	Nghiệm thu
FR01	Sắp xếp qua registry chung	Output đúng, không đổi input
FR02–03	Generator và seed	5 phân phối, lặp lại được
FR04	Đo thời gian	Warm-up, median, min, std, CV
FR05–06	Đếm và đo nhớ	Lượt độc lập, quy ước minh bạch
FR07	CSV và metadata	Có status, seed, hash, môi trường
FR08–10	CLI	Một / tất cả thuật toán, kiểm tra tham số
FR11	Cap và timeout	Ghi skipped/timeout với lý do
FR12	Tái tạo và biểu đồ	Một lệnh sinh CSV, bảng và hình

Phi chức năng gồm độ đúng, tính ổn định đối với thuật toán có cam kết, tái lập dữ liệu, công bằng phép đo, giới hạn thời gian, độ phủ kiểm thử và khả năng đọc mã. Chi tiết tiêu chí và test tương ứng nằm trong docs/SRS_PROTOCOL.md và docs/TEST_PLAN.csv.

Bảng 3 Phân công theo Plan

TV	Trách nhiệm theo Plan	Tệp chính
TV1	Yêu cầu, Insertion/Merge, tích hợp và báo cáo	algorithms.py, SRS, báo cáo
TV2	Quick/Heap, bộ đếm, đối chiếu lý thuyết	algorithms.py, instrumented.py, E3
TV3	Generator, harness, charts và số liệu	data_generator.py, benchmark.py, plots.py
TV4	Kiểm thử, CLI, CI, định dạng và slide	tests/, cli.py, workflow, slide

Bảng mô tả trách nhiệm được giao. Bằng chứng đóng góp cá nhân phải là commit, PR, review hoặc biên bản do thành viên thực hiện thật, không suy ra chỉ từ tên người được giao trong Plan.

Chương 2 Cơ sở lý thuyết và thiết kế

2.1 Độ phức tạp và thuộc tính

Ký hiệu O biểu diễn chặn trên tiệm cận, Ω biểu diễn chặn dưới và Θ biểu diễn chặn chặt hai phía. Phân tích tốt nhất, trung bình và xấu nhất là các cách chọn đầu vào, khác với bản thân ký hiệu tiệm cận. Với khóa phân biệt và mô hình dựa trên so sánh, cây quyết định cần phân biệt n! hoán vị, nên chiều cao xấu nhất ít nhất log2(n!)=Ω(n log n). [1]

Stable nghĩa là giữ thứ tự tương đối của những bản ghi có khóa bằng nhau. In-place mô tả bộ nhớ phụ của kernel trên một mảng có sẵn. Adaptive nghĩa là có thể tận dụng trật tự sẵn có. Trong sản phẩm, API luôn tạo danh sách mới nên ngay cả kernel in-place vẫn có chi phí output O(n).

Bảng 4 Đặc tính lý thuyết

Thuật toán	Tốt nhất	Điển hình	Xấu nhất	Phụ trợ kernel	Stable
Insertion	Θ(n)	Θ(n²)	Θ(n²)	O(1)	Có
Merge	Θ(n log n)	Θ(n log n)	Θ(n log n)	O(n)	Có
Quick cuối	Θ(n log n)	Θ(n log n)*	Θ(n²)	O(log n)**	Không
Quick Mo3	Θ(n log n)	Θ(n log n)*	Θ(n²)	O(log n)**	Không
Heap	Θ(n)***	Θ(n log n)	Θ(n log n)	O(1)	Không
Timsort	Θ(n)	O(n log n)	O(n log n)	O(n)	Có

* Giả định phân phối thuận lợi, không phải cam kết với mọi multiset. ** Nhờ xử lý đoạn nhỏ trước bằng stack tường minh; thời gian xấu nhất vẫn bậc hai. *** Với kernel sift dừng sớm trong sản phẩm, input toàn khóa bằng nhau có thể đạt tuyến tính; với khóa phân biệt, trường hợp điển hình vẫn n log n. Không chép máy móc bảng độ phức tạp của một biến thể Heap khác. [1], [2], [5]

2.2 Quy ước phép toán

Một so sánh là một lần đánh giá < giữa hai khóa. Một move là một lần ghi vào ô danh sách. Swap hai vị trí khác nhau tính hai move; tự swap bị bỏ qua. Việc gán biến tạm, tính chỉ số, tạo bản sao ban đầu và khởi tạo buffer không tính. Merge tính cả ghi buffer và ghi về output. Định nghĩa này được giữ nhất quán giữa trace, test và E3.

2.3 Insertion Sort

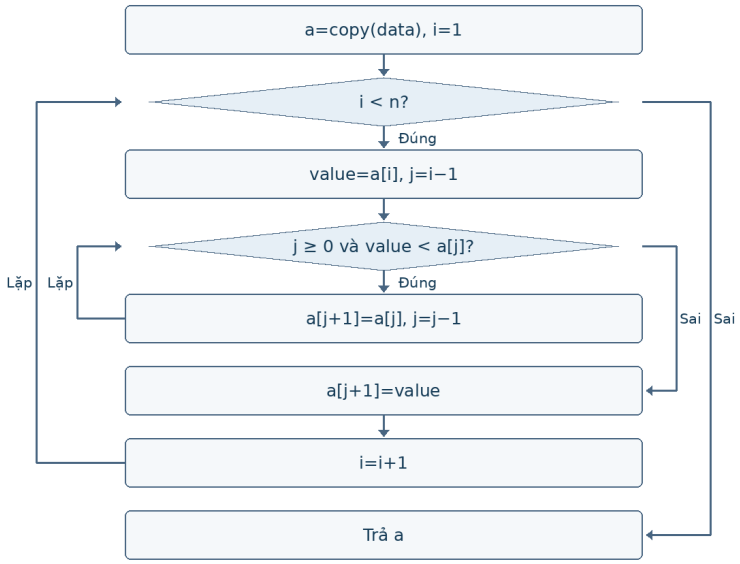
Thuật toán duy trì tiền tố a[0:i] đã sắp xếp. Tại bước i, giữ value=a[i], dịch các khóa lớn hơn value sang phải, rồi chèn value vào chỗ trống. Chỉ dịch khi value < a[j], nên khóa bằng nhau không vượt nhau.

```
a = copy(data)
for i = 1 .. n-1:
    value = a[i]; j = i-1
    while j >= 0 and value < a[j]:
        a[j+1] = a[j]; j = j-1
    a[j+1] = value
return a
```

Bảng 5 Trace Insertion Sort

Bước	Trạng thái trên mẫu
Input	[5a, 2, 9, 1, 5b, 6]
i=1	[2, 5a, 9, 1, 5b, 6]
i=2	[2, 5a, 9, 1, 5b, 6]
i=3	[1, 2, 5a, 9, 5b, 6]
i=4	[1, 2, 5a, 5b, 9, 6]
i=5	[1, 2, 5a, 5b, 6, 9]

Bất biến: trước mỗi vòng i, tiền tố đã có thứ tự và chứa đúng các phần tử ban đầu của tiền tố. Dịch phải giữ nguyên thứ tự những phần tử không bị vượt; chèn value bảo toàn multiset và mở rộng tiền tố đã sắp xếp. Khi i=n, toàn bộ mảng đúng. Trên input giảm dần, C(n)=n(n−1)/2 và M(n)=C(n)+(n−1). Trên input tăng dần, C(n)=M(n)=n−1. [1], [3]



Hình 2.1 Luồng Insertion Sort và vòng lặp dịch phần tử

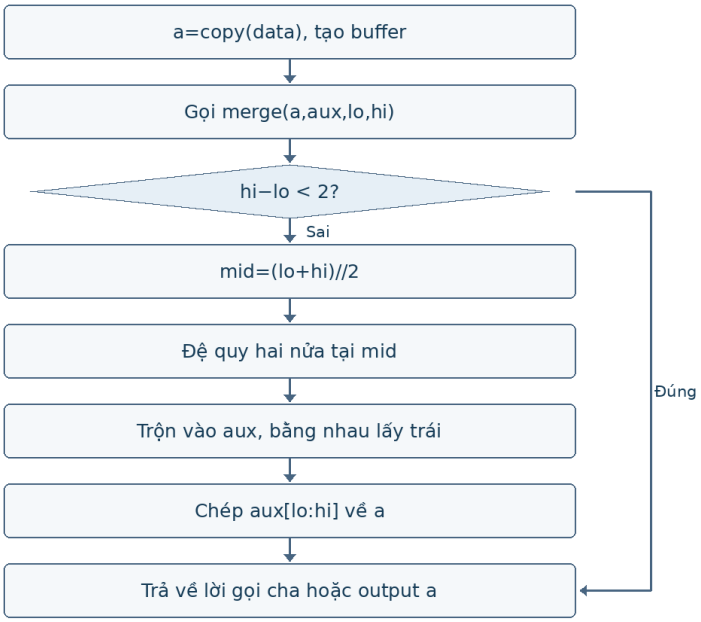
2.4 Merge Sort

Merge chia mảng thành hai nửa, sắp xếp đệ quy rồi trộn. Sản phẩm dùng một buffer O(n) dùng chung và phạm vi [lo,hi), tránh tạo các lát cắt ở mỗi mức. Khi hai khóa bằng nhau, lấy phần tử từ trái trước để giữ ổn định. [2], [4]

```
merge(a, aux, lo, hi):
    if hi-lo < 2: return
    mid = floor((lo+hi)/2)
    merge(a, aux, lo, mid)
    merge(a, aux, mid, hi)
    trộn hai dãy tăng vào aux[lo:hi]
    chép aux[lo:hi] vê`a[lo:hi]
```

Trace: [5a,2,9] trở thành [2,5a,9]; [1,5b,6] đã có thứ tự sau hai nhánh đệ quy. Trộn lần cuối lần lượt lấy 1, 2, 5a, 5b, 6, 9. Việc ưu tiên trái ở hai khóa 5 bảo toàn thứ tự a trước b.

Chứng minh quy nạp: đoạn dài 0 hoặc 1 đã đúng; giả sử hai nửa đúng, vòng trộn luôn chọn khóa nhỏ nhất còn lại nên tạo dãy tăng và không mất phần tử. T(n)=2T(n/2)+Θ(n)=Θ(n log n). Với n là lũy thừa của 2, số lần ghi là 2n log2 n trong cài đặt này; input tăng dần có (n/2)log2 n so sánh. Khi n không là lũy thừa 2, không áp dụng nguyên xi công thức chính xác đó.



Hình 2.2 Luồng Merge Sort và điều kiện dừng đệ quy

2.5 Quick Sort với hai cách chọn pivot

Hai phiên bản dùng cùng partition Lomuto, khác cách chọn pivot. Bản basic lấy a[hi]. Bản cải tiến lấy trung vị của a[lo], a[mid], a[hi] rồi đổi về a[hi]. Bộ đếm bao gồm các so sánh chọn median. Trong partition, các phần tử ≤pivot đưa về trái, >pivot về phải. [1], [2]

```
partition(a, lo, hi):
    pivot=a[hi]; i=lo
    for j=lo .. hi-1:
        if a[j] <= pivot:
            swap(a,i,j); i=i+1
    swap(a,i,hi); return i

Sau partition: lưu đoạn lớn vào stack,
xử lý đoạn nhỏ ngay; bỏ đoạn dài <2.
```

Bất biến partition: a[lo:i]≤pivot và a[i:j]>pivot. Sau vòng quét, đổi pivot về i tạo vị trí chốt đúng. Các phân đoạn nhỏ dần nên thuật toán kết thúc. Stack chỉ chứa các đoạn lớn còn chờ trong lúc xử lý nhánh nhỏ; số tầng đang chờ bị chặn bởi O(log n), không cần thay recursionlimit.

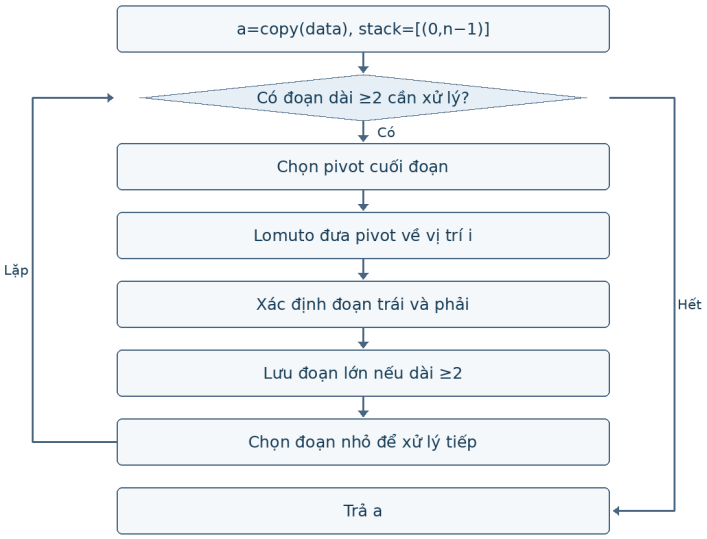
Mẫu pivot cuối: [5,2,9,1,5,6] → partition pivot 6 → [5,2,1,5,6,9]; tiếp tục phần trái với pivot 5 và pivot 1 → [1,2,5,5,6,9]. Với median-of-three, lượt đầu median(5,9,6)=6 nên trùng pivot; ở các lượt sau cấu trúc nhánh có thể khác.

Với input tăng dần và pivot cuối, mỗi partition chỉ loại một phần tử, nên C(n)=n(n−1)/2. Trên input toàn bằng nhau, điều kiện ≤ cũng gây lệch nhánh cho cả hai cách chọn pivot. Median-of-three cải thiện nhiều input có thứ tự nhưng không bảo đảm worst-case O(n log n).

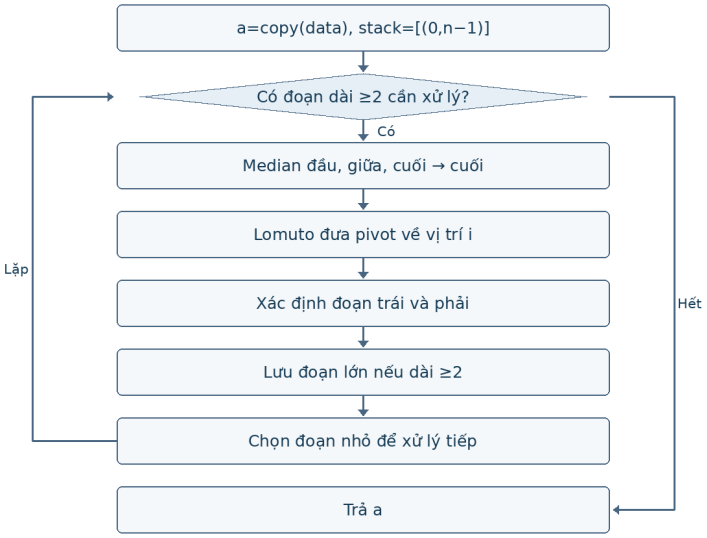
Bảng 6 Đối chiếu hai bản Quick Sort

Tình huống	Quick pivot cuối	Quick median-of-three
Random	Thường gần n log n	Thường cân bằng hơn, thêm phí chọn pivot
Sorted distinct	n(n−1)/2 so sánh	Thường chia cân hơn
Toàn bằng nhau	Lomuto suy biến	Lomuto vẫn suy biến
Tràn đệ quy	Tránh bằng stack	Tránh bằng stack

2.6 Lưu đồ Quick Sort



Hình 2.3 Quick Sort dùng pivot cuối



Hình 2.4 Quick Sort dùng median-of-three

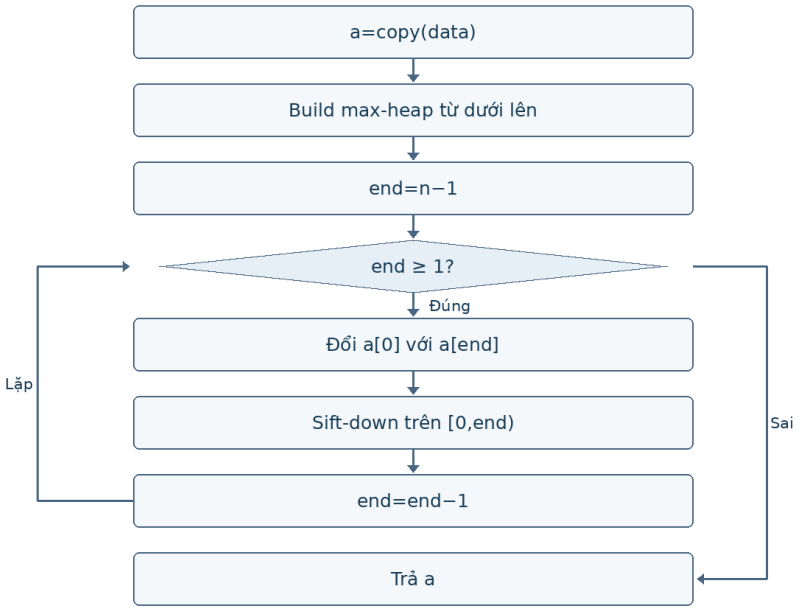
2.7 Heap Sort

Heap Sort xem mảng là cây nhị phân hoàn chỉnh: con trái 2i+1, con phải 2i+2. Max-heap bảo đảm cha không nhỏ hơn con. Build-heap chạy sift-down từ nút trong cuối về gốc; tổng công việc O(n) vì số nút ở độ cao lớn giảm theo cấp số nhân. Mỗi lần đổi gốc với phần tử cuối heap rồi sift-down, một cực đại được đưa vào hậu tố đã sắp xếp. [5]

```
a=copy(data)
for root=floor(n/2)-1 down to 0: sift(root,n)
for end=n-1 down to 1:
    swap(a,0,end)
    sift(0,end)
return a
```

Trace heap từ [5,2,9,1,5,6]: build → [9,5,6,1,2,5]. Sau mỗi lần rút max: [6,5,5,1,2 | 9], [5,2,5,1 | 6,9], [5,2,1 | 5,6,9], [2,1 | 5,5,6,9], [1 | 2,5,5,6,9]. Dấu | phân cách heap còn xử lý với hậu tố đã cố định.

Bất biến: trước mỗi lần rút, tiền tố là max-heap và hậu tố đã tăng dần, chứa các phần tử lớn nhất. Rút gốc đặt đúng phần tử tiếp theo; sift khôi phục heap. Kernel dùng bộ nhớ phụ O(1), nhưng API non-mutating vẫn cần output O(n). Swap xa có thể đảo thứ tự khóa bằng nhau nên không ổn định.



Hình 2.5 Heap Sort với build-heap và vòng sort-down

2.8 Baseline Timsort và thiết kế phần mềm

Python sorted() trả danh sách mới, có tính ổn định và tận dụng các đoạn dữ liệu đã có thứ tự. Họ thuật toán Timsort phát hiện runs, kéo dài đoạn ngắn bằng insertion và trộn; galloping có thể giảm công việc khi một run thắng liên tiếp. Chi tiết chính sách trộn phụ thuộc phiên bản CPython; vì vậy baseline được nhận diện bằng đúng phiên bản trong metadata, không tự cài lại rồi gọi đó là sorted(). [6]

Thời gian sorted() trên số nguyên tận dụng mã C và các tối ưu kiểu dữ liệu. Lượt đếm dùng wrapper Key nên đường tối ưu nội bộ có thể khác. Số comparisons của baseline là số lần gọi __lt__ trên wrapper, còn runtime là số nguyên thuần; hai chỉ số không dùng để suy ra hằng số thời gian cho một phép so sánh chung giữa C và Python.

2.8.1 Module và luồng dữ liệu

Bảng 7 Vai trò các module

Module	Vai trò	Đầu ra
algorithms.py	Kernels và registry 6 thuật toán	Sorted copy
instrumented.py	Key và CountedList dùng chung kernel	Output, comparisons, moves
data_generator.py	5 phân phối, RNG cục bộ	Danh sách số nguyên
benchmark.py	Warm-up, timing, memory, timeout, CSV	raw / summary / metadata
plots.py	Đọc CSV, tính growth và vẽ hình	PNG, SVG, bảng phân tích
cli.py	Kiểm tra tham số, demo và pipeline	Bảng console, tệp kết quả

2.8.2 Luồng CLI

Người dùng chọn demo, run hoặc plots. Demo nhận algorithm, distribution, n và seed; kiểm tra hợp lệ trước khi sinh mảng. Mỗi cấu hình chạy trong process riêng có giới hạn thời gian. Kết quả được gom thành median và xếp tăng dần; CLI in mẫu sắp xếp thực từ thuật toán đã chạy thành công. Nếu tất cả bị cap, trả mã 1; tham số sai trả mã 2.

2.8.3 Tách các vùng đo

Timing gọi API bằng list thường. Count dùng lại kernel với lớp bọc và chỉ thực hiện ngoài timing. Memory chạy API thường dưới tracemalloc trong lượt riêng. Copy nội bộ nằm trong thời gian và bộ nhớ vì là một phần hợp đồng non-mutating. Sinh dữ liệu, kiểm tra kết quả, ghi CSV và spawn process nằm ngoài timer. [7], [8], [9]

Chương 3 Thực nghiệm và kết quả

3.1 Môi trường và tính nhận diện

Bảng 8 Môi trường đo

Thuộc tính	Giá trị ghi nhận
CPU	AMD EPYC 9V74 80-Core Processor
Python	3.12.14 (main, Aug 25 2026, 14:00:49) [Clang 22.1.3]
OS	Linux-6.18.44-x86_64-with-glibc2.39
CPU logic	9
Thời điểm UTC	2026-09-29T10:20:54.823295+00:00
Nguồn điện / power mode	Không kiểm soát được trong container
RAM	Không đo RSS; không có số RAM máy chuẩn được xác nhận
Nguồn số liệu	results/final/raw.csv; summary.csv; metadata.json

Mã nguồn modules được băm SHA256 trước phép đo và lưu trong metadata. Đây là bằng chứng nhận diện code dùng cho số liệu; không giả định có GitHub release đã được nhóm phê duyệt. Thời điểm đo thực tế sớm hơn lịch tuần 9 trong Plan vì đây là bộ sản phẩm kỹ thuật được chuẩn bị trước lịch học phần.

3.2 Ma trận thí nghiệm

Bảng 9 Ma trận thực nghiệm

Thí nghiệm	Thiết kế	Đích quan sát
E1	n=1000,2000,4000,8000,16000; mở rộng 32000	Runtime và tăng trưởng
E2	5 phân phối, đối chiếu tại n=4000	Ảnh hưởng dữ liệu
E3	Đếm cùng cấu hình chính trong lượt riêng	Comparisons và writes
E4	Tracemalloc cùng cấu hình chính	Peak KiB
Mở rộng	n=8,16,32,64,128,256 trên random	Khoảng crossover
Tái lập	6 cấu hình chọn bằng RNG seed=2026	Sai khác median <10%

H1 dự đoán Insertion random tăng bậc hai. H2 dự đoán pivot cuối suy biến trên ordered input. H3 dự đoán các công thức chính xác khớp bộ đếm. H4 dự đoán Merge cần thêm buffer so với các kernel in-place. Đây là giả thuyết được kiểm tra bằng dữ liệu bên dưới.

3.3 Dữ liệu và quy trình đo

Bảng 10 Định nghĩa các phân phối

Phân phối	Định nghĩa chính xác
random	Uniform integer trong [-n,n], seed=42
sorted	0,1,...,n−1, các khóa phân biệt
reversed	n−1,...,1,0, các khóa phân biệt
nearly	Từ sorted, đổi floor(0.01n) cặp vị trí phân biệt
few_unique	Lấy đều từ 0..9; n nhỏ có thể thiếu một số giá trị

Nearly không đồng nghĩa chính xác 1% inversion hay 1% phần tử sai chỗ: các cặp giữa các lượt có thể lặp lại. Input ordered dùng khóa phân biệt, random có thể trùng; do vậy so sánh E2 phản ánh toàn bộ chế độ generator chứ chưa cô lập riêng thứ tự của cùng một multiset.

Mỗi cấu hình dùng một lượt warm-up và R=9 quan sát sau đó. Warm-up hiệu chỉnh số lần gọi trong batch tới khoảng 15 ms, giới hạn 10000 lần. Mỗi quan sát là ms/call trung bình trong batch. Trung vị của 9 quan sát là chỉ số chính; min, std mẫu và CV cũng được giữ. GC tắt trong vùng timer và được khôi phục kể cả khi xảy ra exception. Không chạy các phép đo đồng thời.

Các thuật toán đều sao chép nội bộ nên input truyền vào từng lần gọi vẫn nguyên vẹn. Cùng (n,distribution,seed) phải có cùng input_sha256 cho mọi thuật toán. Đếm và bộ nhớ được lấy trong hai lượt riêng; các cột E3/E4 lặp trên 9 hàng timing để truy vết, không phải 9 mẫu memory độc lập.

3.3.1 Giới hạn cấu hình

Với n>4000, bỏ qua Insertion ngoài sorted, Quick-last trên sorted/reversed, cả hai bản Quick trên few_unique. Cấu hình có status=skipped và lý do, không gán thời gian 0. Timeout 45 giây áp dụng cho toàn configuration worker; thời gian khởi tạo/dọn process nằm ngoài timer. 24 cấu hình bị cap, không có timeout trong bộ cuối.

3.3.2 Công thức phân tích

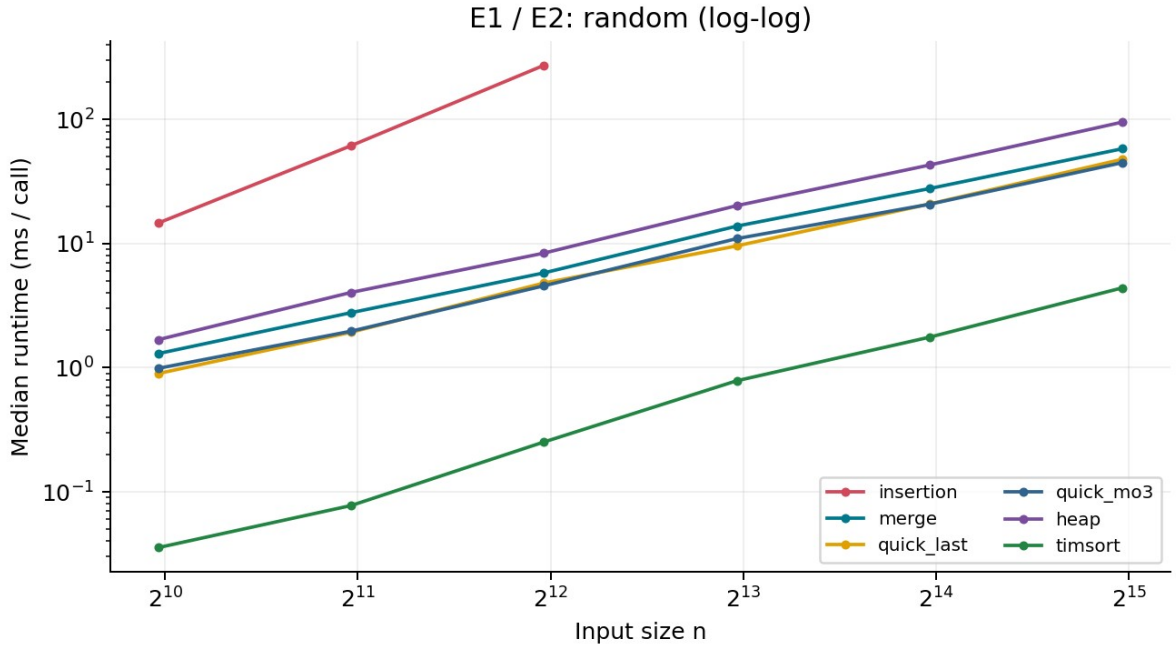
Doubling ratio D(n)=T(2n)/T(n). Với n^2, D gần 4; với n log2 n, D≈2(1+1/log2 n). Hệ số b được fit từ ln(T)=a+b ln(n) bằng bình phương tối thiểu trên các điểm random có số đo. CV=100×s/mean; s là độ lệch chuẩn mẫu của các batch. Sai khác tái lập =100×|median mới−median cũ|/median cũ.

3.4 E1 thời gian theo kích thước

Bảng 11 Thời gian trên random

n	Insertion	Merge	Quick cuối	Quick Mo3	Heap	Timsort
1000	14.667	1.301	0.899	0.989	1.685	0.036
2000	61.605	2.779	1.932	1.970	4.045	0.078
4000	273.558	5.812	4.813	4.575	8.387	0.252
8000	Bỏ qua	13.836	9.588	10.976	20.237	0.785
16000	Bỏ qua	27.781	20.858	20.782	42.998	1.763
32000	Bỏ qua	58.165	47.839	44.928	95.339	4.402

Median ms/call trên random, seed=42. Nguồn: summary.csv, experiment=E1E2. Ô bỏ qua theo cap không được nội suy thành số đo.



Hình 3.1 Thời gian random trên trục log-log

Insertion tăng nhanh nhất và bị giới hạn ở n=4000. Nhóm Merge/Quick/Heap có đường tăng thấp hơn rõ rệt trong miền khảo sát. Timsort có thời gian nhỏ hơn đáng kể, nhưng khoảng cách gồm cả lợi thế cài đặt C. Chi tiết min, std và CV nằm trong summary.csv.

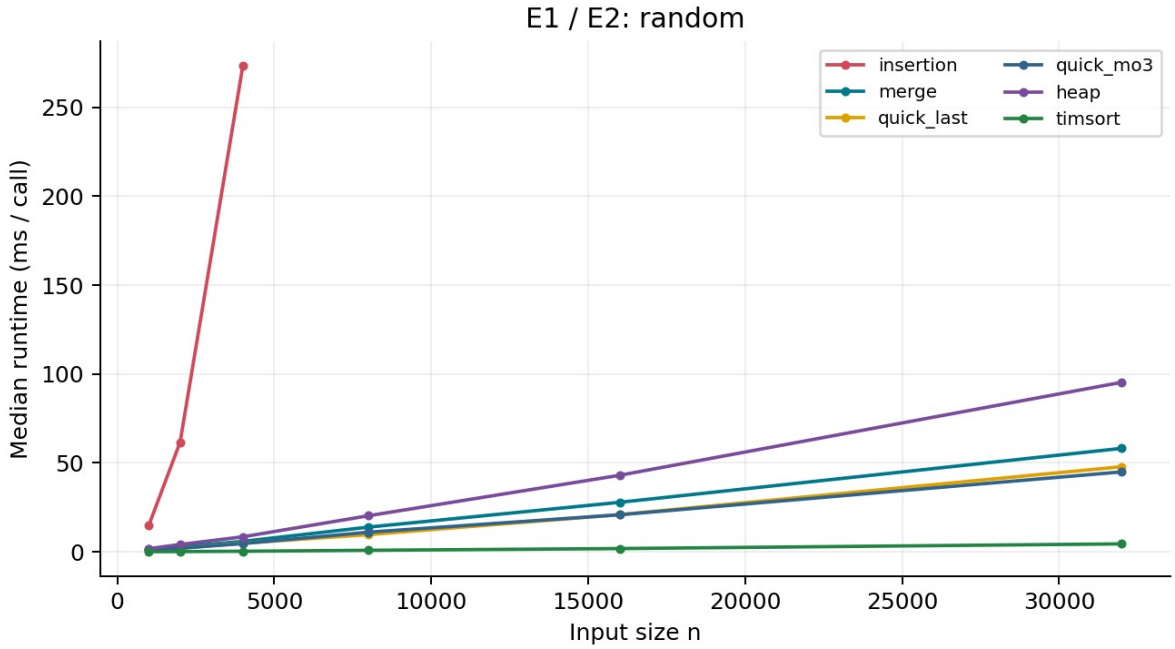
3.5 Tăng trưởng thực nghiệm

Bảng 12 Tăng trưởng thực nghiệm

Thuật toán	Slope b	Khoảng D(n)	Số cỡ đo
Insertion	2.111	4.20–4.44	3
Merge	1.104	2.01–2.38	6
Quick cuối	1.142	1.99–2.49	6
Quick Mo3	1.114	1.89–2.40	6
Heap	1.160	2.07–2.41	6
Timsort	1.426	2.18–3.25	6

Ước lượng từ growth.csv trên random. Insertion có slope 2,111 gần 2; các thuật toán Python thuộc nhóm n log n có slope khoảng 1,10–1,16. Chênh lệch quanh lý thuyết phản ánh hằng số, cache, overhead và nhiễu; không dùng slope hữu hạn để khẳng định lớp độ phức tạp mới.

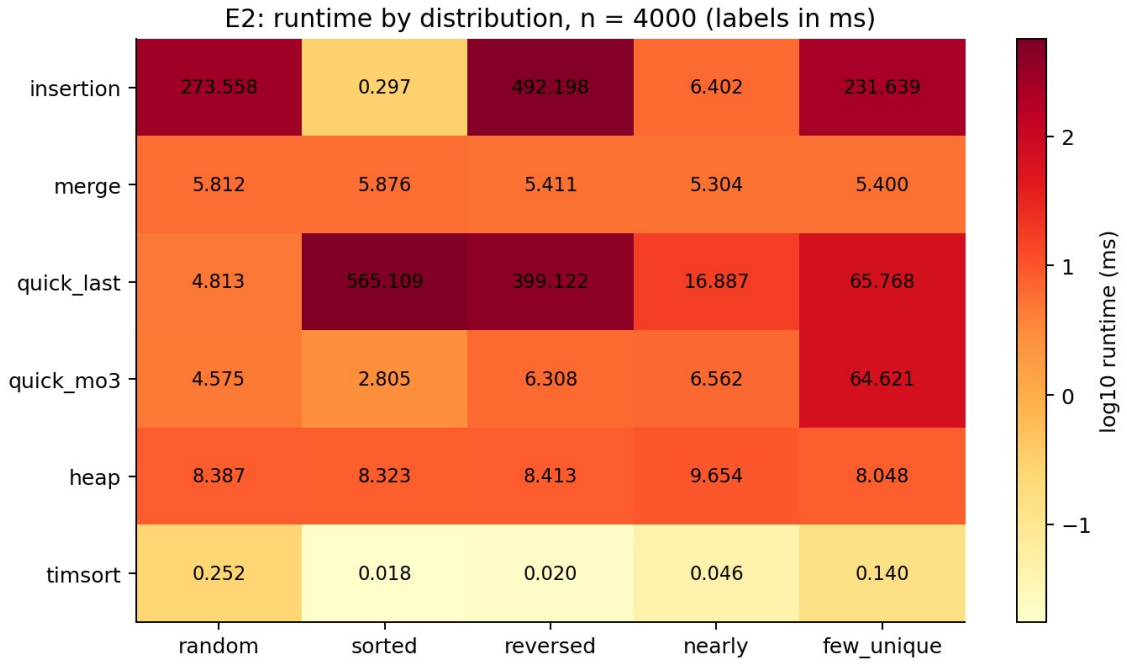
Slope Timsort là 1,426 trong bộ này. Không suy ra sorted() có độ phức tạp Θ(n^1.426). Thời gian rất nhỏ ở cỡ thấp chịu ảnh hưởng overhead, các chuyển tiếp cache và trật tự dữ liệu. Một seed và sáu cỡ là giới hạn quan trọng; cần thêm seed, cỡ lớn và kiểm soát môi trường để xác định xu hướng ổn định hơn.



Hình 3.2 Trục tuyến tính làm rõ mức chi phí tuyệt đối

H1 được hỗ trợ về xu hướng: Insertion random tăng gần bậc hai, còn các thuật toán chia để trị và heap tăng chậm hơn. Chênh lệch giữa các giá trị D(n) không chỉ do bản thân thuật toán mà còn do đo trên hệ thống không được cô lập tài nguyên.

3.6 E2 ảnh hưởng của phân phối



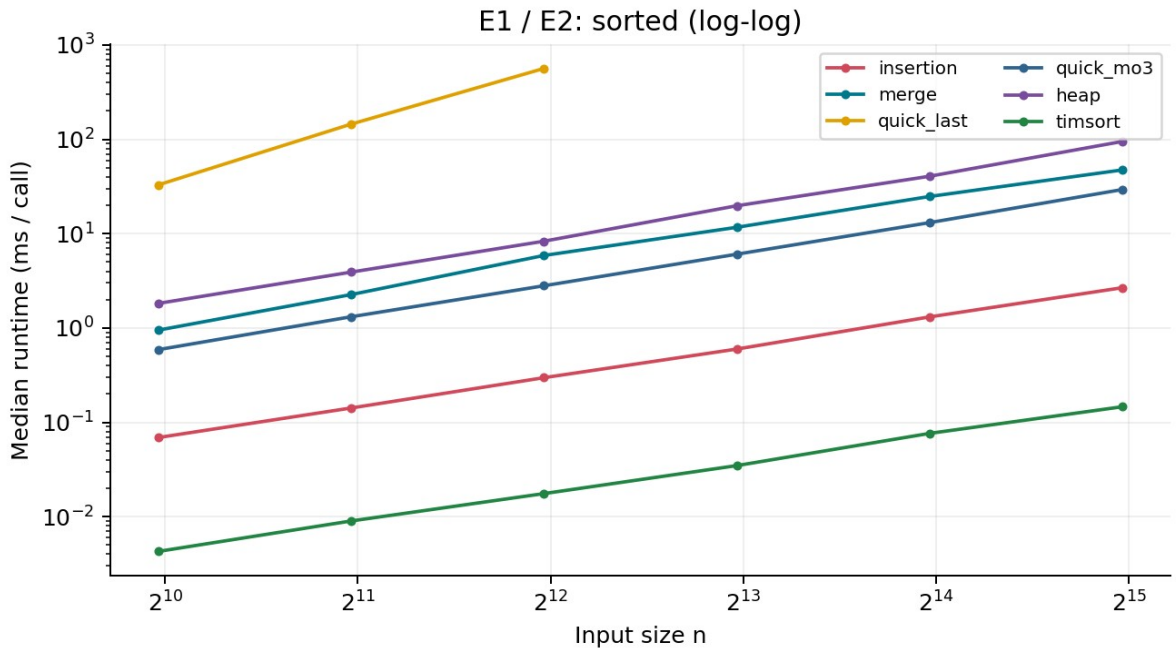
Hình 3.3 Median tại n=4000; nhãn ô là ms, màu dùng log10

Bảng 13 Ảnh hưởng phân phối tại n bằng 4000

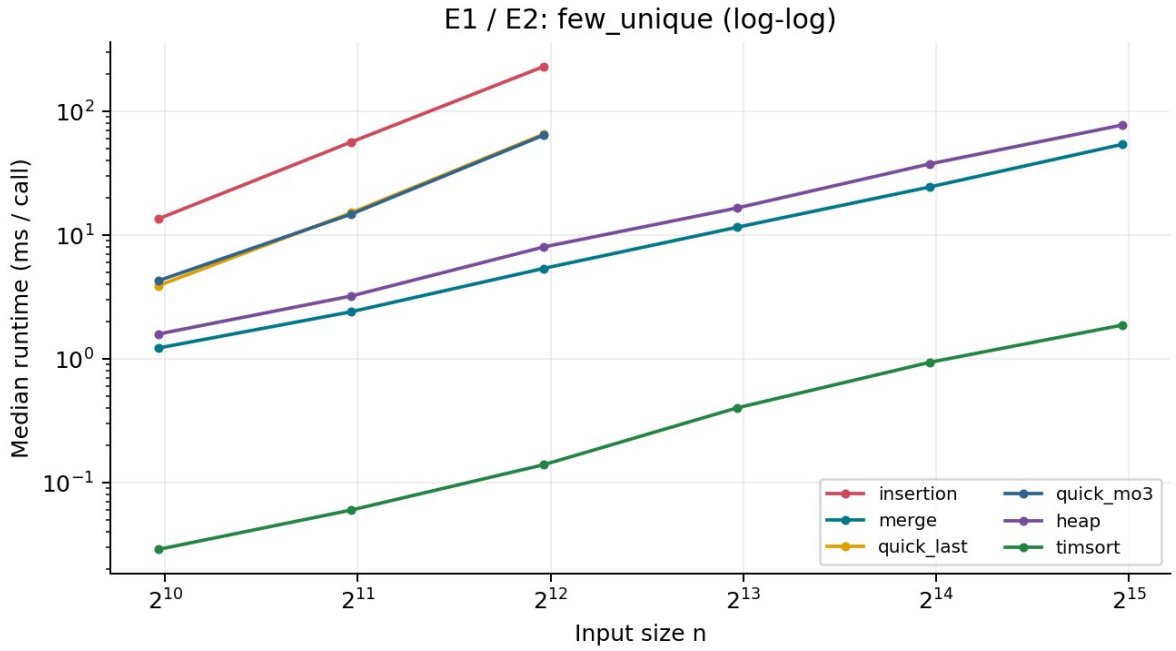
Thuật toán	Nhanh nhất	Chậm nhất	Max / min
Insertion	sorted	reversed	1,655.14
Merge	nearly	sorted	1.11
Quick cuối	random	sorted	117.42
Quick Mo3	sorted	few_unique	23.04
Heap	few_unique	nearly	1.20
Timsort	sorted	random	14.37

Insertion sorted chỉ cần kiểm tra n−1 lần, trong khi reversed cần n(n−1)/2 lần. Quick-last chậm nhất trên sorted ở bộ này; Quick Mo3 cải thiện ordered distinct nhưng vẫn chậm trên few_unique. Merge thay đổi nhỏ giữa các chế độ, vì luôn chia và trộn. Chênh lệch nhỏ giữa các phân phối của Merge/Heap không nên được khái quát thành quy luật tuyệt đối do một số CV cao.

3.7 Suy biến Quick Sort và dữ liệu gần có thứ tự



Hình 3.4 Sorted distinct làm Quick pivot cuối tăng bậc hai



Hình 3.5 Khóa trùng gây bất lợi cho Lomuto ở cả hai bản Quick

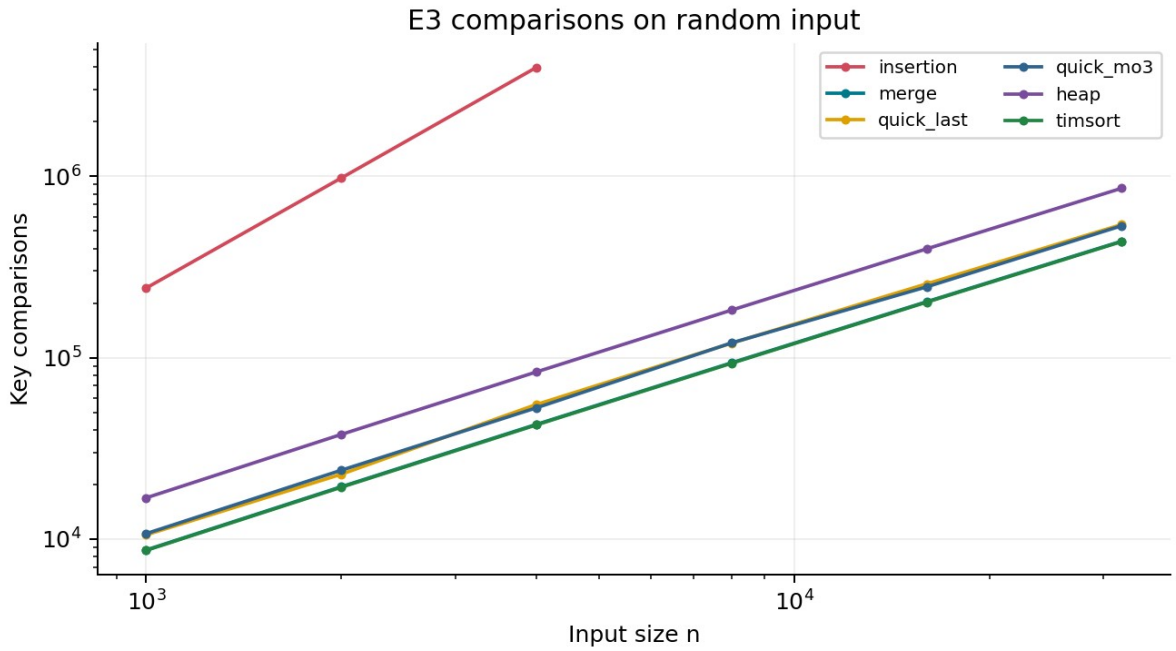
Tại n=4000, Quick-last trên sorted mất 565,109 ms so với Quick Mo3 2,805 ms. Nhưng trên few_unique, Quick Mo3 mất 64,621 ms, lớn hơn rõ rệt random 4,575 ms. H2 được hỗ trợ. Đổi chiến lược pivot không thay thế partition ba phía khi muốn xử lý hiệu quả nhiều khóa trùng.

3.8 E3 số phép toán

Bảng 14 Phép đếm tại n bằng 4000

Thuật toán	So sánh random	Writes random	So sánh sorted
Insertion	3,983,428	3,983,434	3,999
Merge	42,826	95,808	23,728
Quick cuối	55,140	52,626	7,998,000
Quick Mo3	52,877	51,908	44,011
Heap	83,409	88,666	86,576
Timsort	42,566	N/A	3,999

Phép đếm tại n=4000. Timsort moves=N/A vì không quan sát được ghi trong C. Quick-last sorted có 0 writes do tất cả swap là tự swap và được bỏ qua; điều đó không làm giảm số so sánh bậc hai.



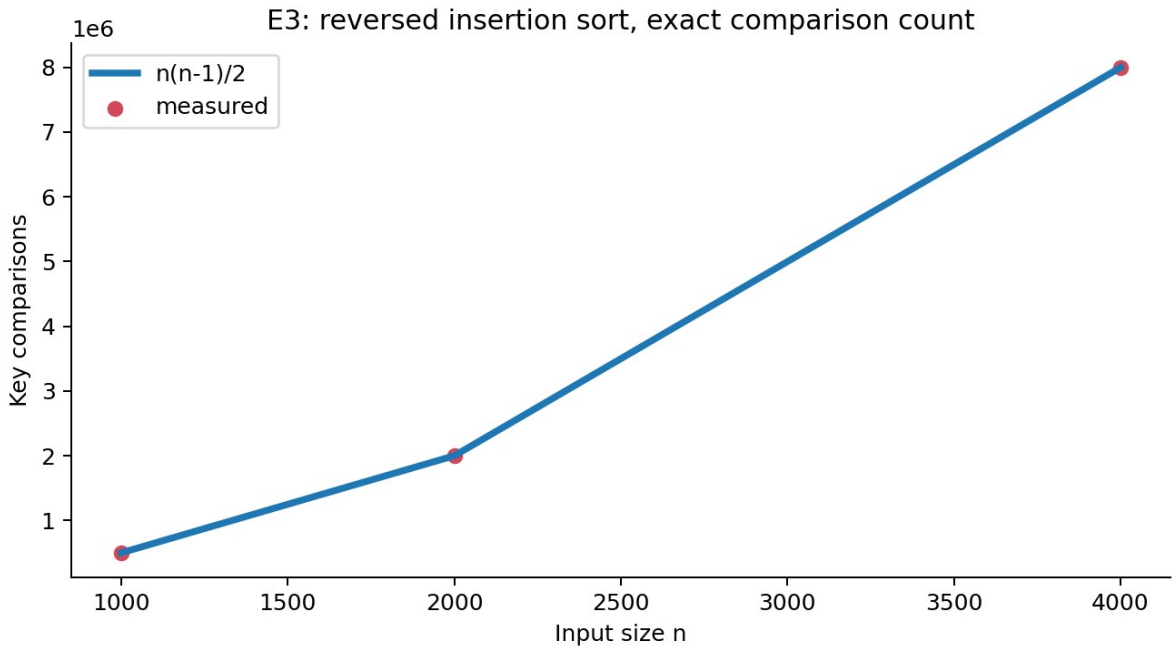
Hình 3.6 Số so sánh trên random

Merge có 42.826 so sánh random tại n=4000, Quick Mo3 52.877, Heap 83.409. Số so sánh ít hơn không đồng nghĩa luôn nhanh hơn: Merge còn ghi buffer và copy lại, Quick có phí gọi swap, Heap có truy cập vị trí xa. Cùng bậc n log n nhưng hằng số, bộ nhớ và mã Python khác nhau.

3.9 Đối chiếu công thức và số lần ghi

Bảng 15 Đối chiếu công thức

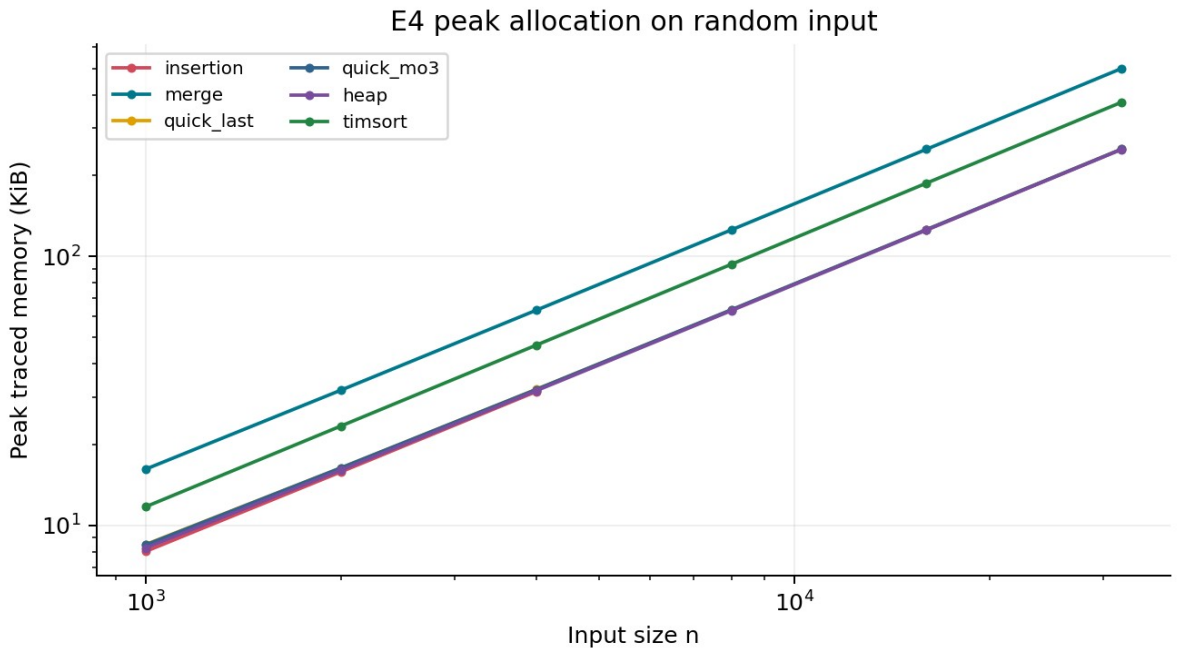
Trường hợp	n	Lý thuyết C	Đo C	Kết luận
Insertion reversed	1000	499500	499500	Khớp
Insertion reversed	2000	1999000	1999000	Khớp
Insertion reversed	4000	7998000	7998000	Khớp
Quick-last sorted	1000	499500	499500	Khớp
Quick-last sorted	2000	1999000	1999000	Khớp
Quick-last sorted	4000	7998000	7998000	Khớp



Hình 3.7 Công thức n(n−1)/2 khớp Insertion reversed

Tại n=4000, Insertion reversed có 7.998.000 so sánh và 8.001.999 writes; chênh lệch 3.999 là lần ghi chèn cuối mỗi vòng ngoài. Merge random có 95.808 writes, bao gồm cả buffer và copy về output. H3 được hỗ trợ bởi dữ liệu nguyên chính xác và test công thức ở các n nhỏ; số liệu này không chịu nhiễu đồng hồ.

3.10 E4 đỉnh bộ nhớ



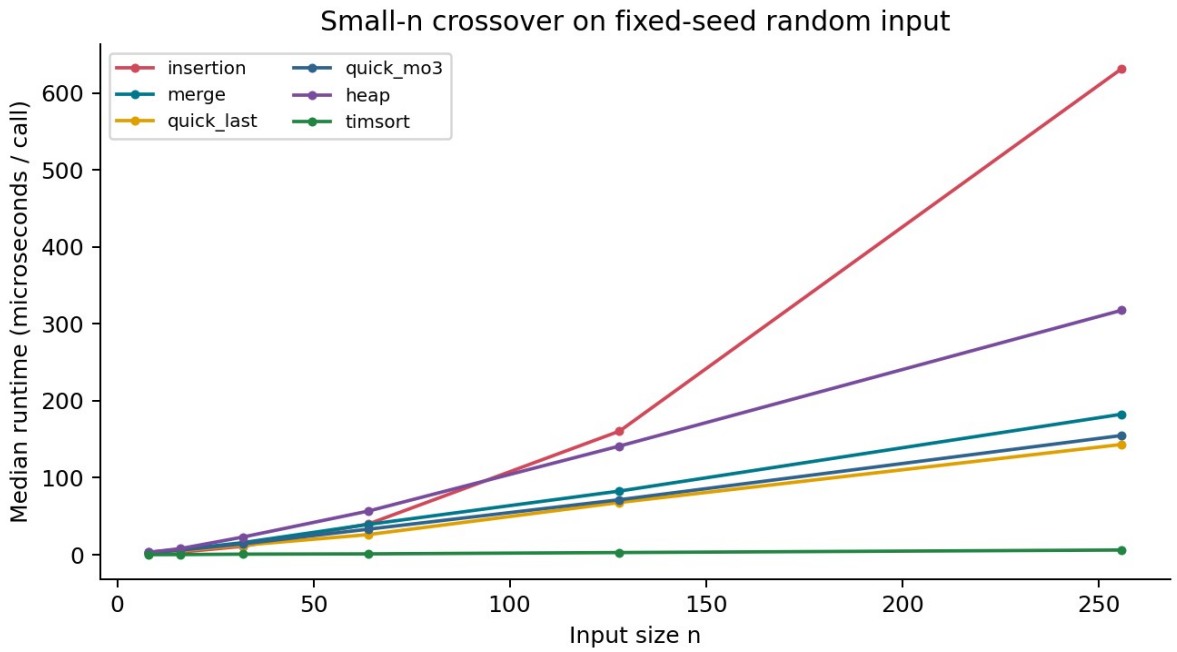
Hình 3.8 Peak allocation trên random; bao gồm danh sách output

Bảng 16 Đỉnh bộ nhớ trên random

Thuật toán	n=4000 KiB	n=16000 KiB
Insertion	31.45	Bỏ qua
Merge	63.14	250.70
Quick cuối	32.05	125.88
Quick Mo3	32.03	126.00
Heap	31.69	125.44
Timsort	46.82	187.54

Merge sử dụng buffer cùng kích thước output nên allocation cao hơn nhóm Insertion/Quick/Heap. H4 được hỗ trợ trong cài đặt. Số đo gồm overhead đối tượng Python và frame; không nên lấy chênh lệch nhỏ giữa Quick và Heap để kết luận một công thức RAM chính xác. Input đã tồn tại trước tracing không nằm trong peak này. [8]

3.11 Thí nghiệm kích thước nhỏ



Hình 3.9 Small-n trên random seed=42; đơn vị microsecond/call

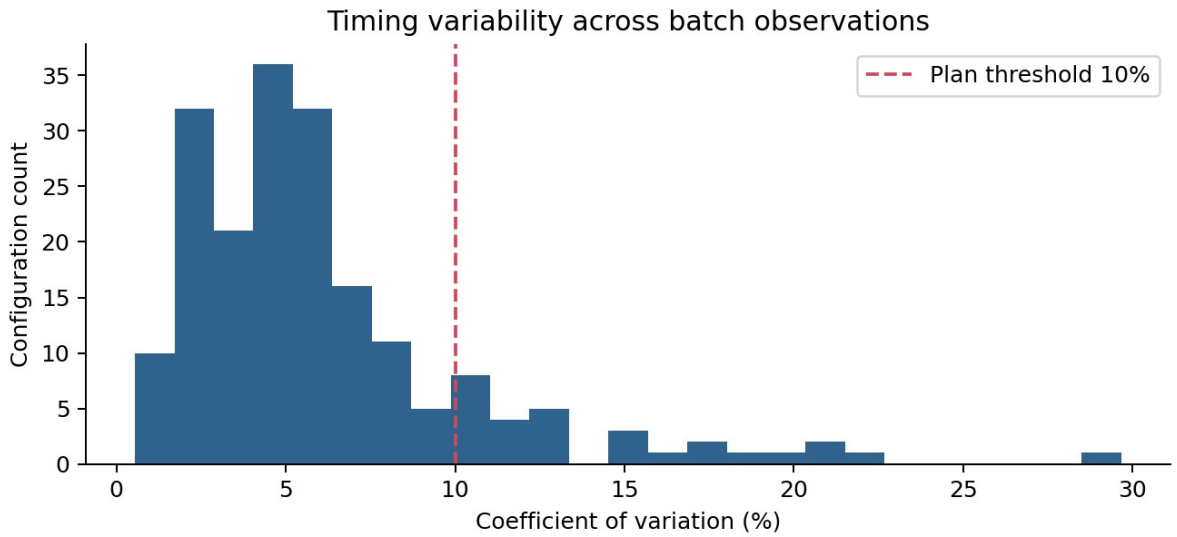
Bảng 17 Thời gian ở kích thước nhỏ

n	Insertion µs	Merge µs	Quick Mo3 µs
8	1.15	3.08	2.26
16	3.36	6.80	5.99
32	10.97	15.70	14.14
64	39.95	39.40	33.23
128	160.62	82.76	71.50
256	631.76	182.65	155.01

Trong bộ đo này, Insertion nhanh hơn Merge và Quick Mo3 ở n=8,16,32. Tại n=64, Quick Mo3 đã nhanh hơn và Merge xấp xỉ Insertion. Khoảng giao quan sát nằm giữa 32 và 64; có thể chọn 32 làm ứng viên cutoff ban đầu cho thí nghiệm hybrid tiếp theo, không phải giá trị tối ưu đã chứng minh.

Timsort vẫn nhanh trong miền nhỏ nhờ mã C và tối ưu nội bộ. Đề tài chưa triển khai hybrid sort; phần mở rộng chỉ cung cấp bằng chứng để đề xuất hướng phát triển. Cần khảo sát nhiều seed và lưới n dày hơn quanh 32–64 trước khi cố định cutoff.

3.12 Độ ổn định và tái lập phép đo



Hình 3.10 Phân bố CV của 192 cấu hình có đo

Có 28/192 cấu hình vượt CV=10%, CV lớn nhất 29.66%. Do đó gate CV<=10% ở tuần 7 chưa đạt toàn bộ. Báo cáo giữ nguyên các quan sát, không loại các lần chậm để cải thiện chỉ số. Cần đo lại trên máy chuẩn khi đã kiểm soát nguồn điện, tải nền và thời gian batch.

Bảng 18 Kiểm tra tái lập

Thuật toán / dạng	n	Median cũ ms	Median mới ms	Lệch %
heap / sorted	16000	40.628	42.643	4.96
timsort / reversed	32000	0.160	0.155	3.28
heap / sorted	2000	3.914	3.704	5.38
quick_last / few_unique	2000	15.201	15.925	4.76
quick_mo3 / random	16000	20.782	19.944	4.03
timsort / few_unique	8000	0.401	0.395	1.61

Sáu cấu hình chọn bằng seed=2026 đều đạt sai khác median <10%. Kết quả này chứng minh khả năng tái lập trên mẫu đã chọn, không đồng nghĩa tất cả 192 cấu hình đều đạt. CV đo dao động bên trong một đợt; recheck đo chênh median giữa hai đợt, nên hai kết quả không mâu thuẫn.

3.13 Kiểm thử và độ tin cậy chương trình

Test Plan có 87 dòng thiết kế, trong đó 72 ca thuật toán là 6 thuật toán ×12 dạng đầu vào. Pytest mở rộng tham số thành 137 test items. Hypothesis chạy 1.000 ví dụ của một test property; mỗi ví dụ kiểm tra cả sáu thuật toán và bản đếm. Số ví dụ sinh tự động không được cộng nhầm thành 1.000 pytest items.

Bảng 19 Các nhóm kiểm thử

Nhóm	Nội dung kiểm tra	Bằng chứng
Unit	Rỗng, đơn, âm, trùng, ordered, số nguyên lớn	72 ca tham số thuật toán
Property	Tăng dần, hoán vị, không đổi input, bản đếm khớp	1000 ví dụ Hypothesis
Stability	Record có key và serial; khóa bằng nhau	Insertion, Merge, Timsort
Instrumentation	Công thức Insertion, Quick, Merge lũy thừa 2	test_exact_counts, test_merge_formula
Integration	Generator → harness → CSV, đủ cấu hình	test_matrix_and_cli
Robustness	Input sai, cap, timeout, GC/tracing cleanup	CLI và isolation tests

Bảng 20 Độ phủ dòng lõi

Module lõi	Statements	Covered	Coverage
__init__.py	1	1	100.00%
algorithms.py	92	92	100.00%
benchmark.py	151	145	96.03%
cli.py	57	57	100.00%
data_generator.py	21	21	100.00%
instrumented.py	37	37	100.00%

Kết quả: 137 passed; coverage dòng 98.33% đạt mục tiêu >=90%. Coverage loại trừ plots.py và __main__.py theo cấu hình đã công bố, không phải độ phủ nhánh toàn bộ sản phẩm. Pipeline và charts đã chạy end-to-end ngoài coverage. Flake8 không báo lỗi trong sortinglab và tests. Log gốc ở evidence/.

Workflow GitHub Actions được cung cấp sẵn, nhưng chưa có bằng chứng đã chạy CI trên repo trực tuyến của nhóm. Phép kiểm thử tự động cũng không thay thế yêu cầu review chéo bởi thành viên khác tác giả trong Plan.

3.14 Trả lời câu hỏi nghiên cứu và khuyến nghị

RQ1 Tăng trưởng theo n

Trong miền đo, Insertion random có slope 2,111 và D(n) khoảng 4,20–4,44, phù hợp chi phí bậc hai. Merge, hai Quick trên random và Heap có slope khoảng 1,10–1,16, phù hợp xu hướng n log n. Công thức đếm chính xác giúp phân biệt tăng trưởng thuật toán với nhiễu thời gian. Không ngoại suy các ô cap thành số đo thực.

RQ2 Ảnh hưởng phân phối

Insertion hưởng lợi lớn từ sorted và nearly, Quick pivot cuối bị bất lợi với ordered distinct. Median-of-three giảm tình huống này nhưng Lomuto vẫn nhạy với khóa trùng. Merge ít nhạy hơn với các phân phối đã xét. Timsort tận dụng runs có sẵn. E2 chưa tách riêng hiệu ứng thứ tự khỏi thay đổi multiset, vì generator ordered và random được định nghĩa khác nhau.

RQ3 Lựa chọn thuật toán

Bảng 21 Khuyến nghị lựa chọn

Nhu cầu	Lựa chọn hợp lý	Điều kiện
Sản phẩm Python thực tế	sorted() / list.sort()	Chọn theo yêu cầu giữ input; thư viện ổn định
Mảng nhỏ trong cài đặt tự viết	Insertion	Ứng viên cutoff 32, cần khảo sát thêm
Ổn định và đảm bảo worst-case	Merge	Chấp nhận buffer O(n)
Giới hạn phụ trợ kernel	Heap	Không cần stable; API copy vẫn O(n)
Khóa ngẫu nhiên, triển khai học thuật	Quick Mo3	Không bảo đảm worst-case; chú ý duplicates
Nhiều khóa trùng hoặc đối kháng	Timsort hoặc nghiên cứu 3-way / introsort	Không mặc định Mo3 đã xử lý mọi suy biến

Giới hạn và hướng cải tiến

Các hạn chế gồm một môi trường container, một seed chính, CV chưa đạt toàn bộ, cỡ lớn bậc hai bị cap, baseline C không ngang điều kiện ngôn ngữ và moves Timsort không quan sát được. Hướng phát triển là nhiều seed với khoảng tin cậy, tách multiset và order, partition ba phía, hybrid cutoff, CPU affinity nếu có quyền và tự động sinh lại tài liệu từ bộ đo mới.

Kết luận

Bài tập lớn tạo được một sản phẩm CLI hoàn chỉnh để sắp xếp, so sánh và tái tạo phép đo. Sáu phiên bản dùng cùng API không thay đổi input; generator, instrumentation, harness, biểu đồ và kiểm thử được tách mô-đun rõ ràng. Bộ số liệu gồm E1–E4 và thí nghiệm small-n, có status bỏ qua, seed, input hash và source hash để truy vết.

Các kết quả hỗ trợ nhận định lý thuyết về Insertion bậc hai, Quick suy biến và chi phí n log n của nhóm còn lại trong trường hợp điển hình. Điểm khác biệt quan trọng là độ phức tạp không đủ để quyết định tốc độ ở n nhỏ; phân phối, cài đặt và chi phí ngôn ngữ có vai trò đáng kể. Crossover 32–64 trong bộ đo là một ví dụ cụ thể.

Chất lượng chức năng được hỗ trợ bởi 137 test và coverage lõi 98,33%. Mẫu recheck đạt ngưỡng sai khác 10%, nhưng gate CV toàn ma trận còn 28 cấu hình chưa đạt. Bước hoàn thiện thực nghiệm trên máy chuẩn cần ưu tiên xử lý nhiễu này trước khi nhóm khẳng định đã đáp ứng toàn bộ gate của Plan.

Trước khi nộp, nhóm cần điền thông tin hành chính, thực hiện review chéo, ghi biên bản thật, tạo bằng chứng GitHub và tập dượt. Những hoạt động đó được tách khỏi trạng thái hoàn thành của mã nguồn và tài liệu, nhằm giữ hồ sơ báo cáo chính xác và có thể kiểm chứng.

Danh mục tệp kết quả

Bảng 22 Danh mục kết quả

Tệp / thư mục	Nội dung
submission/	Báo cáo Word/PDF, slide
sortinglab/ và tests/	Mã nguồn, kiểm thử
results/final/	raw, summary, E3, E4, metadata, growth, recheck
results/final/figures/	22 hình PNG và SVG từ CSV
docs/	SRS, protocol, test plan, handoff, Q&A, lưu đồ
evidence/	Log kiểm thử, coverage, lint, demo và benchmark
plan/	5 workbook gốc để đối chiếu

Tài liệu tham khảo

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest và C. Stein, Introduction to Algorithms, 4th ed. MIT Press, 2022. https://mitpress.mit.edu/9780262046305/introduction-to-algorithms/

[2] R. Sedgewick và K. Wayne, Algorithms, 4th ed. Addison-Wesley, 2011. Tài liệu tác giả: https://algs4.cs.princeton.edu/20sorting/

[3] D. E. Knuth, The Art of Computer Programming, Vol. 3: Sorting and Searching, 2nd ed. Addison-Wesley, 1998. https://www-cs-faculty.stanford.edu/~knuth/taocp.html

[4] R. Sedgewick và K. Wayne, “Mergesort,” Algorithms, 4th ed., online companion. https://algs4.cs.princeton.edu/22mergesort/ (truy cập 29/09/2026).

[5] R. Sedgewick và K. Wayne, “Heap,” Algorithms 4/e Java API. https://algs4.cs.princeton.edu/code/javadoc/edu/princeton/cs/algs4/Heap.html (truy cập 29/09/2026).

[6] Python Software Foundation, “Sorting Techniques,” Python Documentation. https://docs.python.org/3/howto/sorting.html (truy cập 29/09/2026).

[7] Python Software Foundation, “time — Time access and conversions,” Python Documentation. https://docs.python.org/3/library/time.html (truy cập 29/09/2026).

[8] Python Software Foundation, “tracemalloc — Trace memory allocations,” Python Documentation. https://docs.python.org/3/library/tracemalloc.html (truy cập 29/09/2026).

[9] Python Software Foundation, “gc — Garbage Collector interface,” Python Documentation. https://docs.python.org/3/library/gc.html (truy cập 29/09/2026).

[10] Nhóm thực hiện, “Plan.xlsx và Nhiem_vu_TV1–TV4.xlsx,” bộ kế hoạch học phần do người dùng cung cấp, 2026.

Các diễn giải thuật toán và chứng minh trong báo cáo gắn với mã nguồn cụ thể của sản phẩm. Tài liệu tham khảo cung cấp nền tảng; không sao chép nguyên bảng của biến thể khác để thay cho việc phân tích cài đặt hiện tại.

Phụ lục A Hướng dẫn chạy và demo

A1 Cài đặt

Dùng Python 3.12; mở Terminal tại thư mục SortingLab sau khi giải nén. Trên Windows, có thể dùng py -3.12 thay python khi tạo môi trường.

```
python -m venv .venv
.venv\Scripts\python.exe -m pip install -r requirements.txt
.venv\Scripts\python.exe -m sortinglab demo --n 1000
```

Linux/macOS: source .venv/bin/activate, sau đó dùng python. Lõi CLI không cần thư viện ngoài; vẽ hình cần Matplotlib và NumPy. Các thư viện còn lại phục vụ test/lint.

A2 Ba tình huống demo

```
python -m sortinglab demo --n 1000 --distribution random
python -m sortinglab demo --n 2000 --distribution sorted
python -m sortinglab demo --n 2000 --distribution few_unique
python -m sortinglab demo --n -1
```

Demo lần lượt minh họa so sánh cơ bản, suy biến pivot cuối, khóa trùng và xác thực tham số. Các log thực có trong evidence/demo_random.txt, demo_sorted.txt và demo_few_unique.txt. Khi trình chiếu, giải thích đơn vị ms, KiB và N/A ở moves của Timsort.

A3 Tái tạo số liệu và kiểm thử

```
python -m sortinglab run --profile full \
  --out results/reproduced --repeats 9
python -m scripts.reproduce_check --results results/reproduced
python -m pytest --cov --cov-report=term-missing
python -m flake8 sortinglab tests
```

Trên Windows nhập lệnh run trên một dòng, bỏ ký tự nối dòng \. Profile smoke dùng n=32,64 để kiểm tra pipeline; pilot dùng n=1000,2000,4000. Không ghi đè results/final khi chỉ muốn chạy thử. Sau khi đổi code hoặc số đo, cần cập nhật report và slide cùng nhau.

Phụ lục B Kiểm thử và nghiệm thu

Bảng 23 Nghiệm thu theo Plan

Tiêu chí trong Plan	Bằng chứng hiện có	Trạng thái
6 phiên bản + 5 dạng dữ liệu	Registry và generator	Đạt kỹ thuật
>=60 unit cases	72 ca thuật toán; 137 pytest items	Đạt
>=1000 property examples	Hypothesis max_examples=1000	Đạt
Coverage lõi >=90%	coverage.json: 98,33%	Đạt
CV<=10% mọi cấu hình	28/192 cấu hình >10%	Chưa đạt toàn bộ
Tái lập mẫu <10%	6/6 cấu hình đạt	Đạt trên mẫu
>=12 hình	22 hình PNG + SVG	Đạt
>=8 tài liệu	9 nguồn lý thuyết/phương pháp + Plan	Đạt danh mục
CLI + một lệnh tái tạo	README, log demo, run full	Đạt trên Linux
CI xanh, >=6 PR có review	Có workflow; chưa có URL hoạt động	Nhóm thực hiện
Biên bản, 2 buổi tập dượt	Có mẫu và kịch bản	Nhóm thực hiện
Báo cáo, slide và code	Bộ submission và mã nguồn	Có tệp bàn giao

Bảng nghiệm thu không thay thế rubric chính thức của giảng viên. Chưa có tài liệu rubric riêng trong các file được cung cấp; vì vậy không đặt trọng số hay hứa hẹn mức điểm. Test Plan, dữ liệu và các tiêu chí được giữ có thể kiểm tra độc lập.

B1 Cách đọc trạng thái dữ liệu

ok: có phép đo thật; skipped: áp dụng cap trước khi đo; timeout: worker vượt ngân sách cấu hình. Ô trống moves của Timsort là không quan sát được. Không dùng giá trị trống hay skipped như số 0 khi tính trung bình hoặc vẽ log scale. Mỗi E3/E4 là một phép đo riêng cho một cấu hình, dù giá trị lặp trong raw CSV.

Phụ lục C Phân công và hồ sơ bàn giao

Bốn hồ sơ docs/BAN_GIAO_TV1.md đến BAN_GIAO_TV4.md giữ liên kết Wxx-yy với task gốc. Ma trận docs/PLAN_TRACEABILITY.csv có đủ 100 task, owner theo Plan, deliverable yêu cầu và đường dẫn nhận bàn giao. Năm bảng tính trong plan/ giữ nguyên nội dung gốc để nhóm cập nhật trạng thái theo hoạt động thật.

C1 Phân chia 15 phút trình bày

Bảng 24 Phân chia thời gian báo cáo

Khoảng thời gian	Người	Nội dung
0:00–3:45	TV1	Bài toán, RQ, yêu cầu và kiến trúc
3:45–7:30	TV2	Thuật toán, pivot, phép đếm
7:30–11:15	TV3	Dữ liệu, protocol và E1–E4
11:15–15:00	TV4	QA, demo, crossover, tái lập và kết luận

Kịch bản chi tiết và 20 câu trả lời gợi ý nằm trong docs/BAO_VE.md. Nhóm cần tập dượt thực tế, điều chỉnh theo tốc độ nói và dành khoảng dự phòng khi thao tác demo.

C2 Hồ sơ trước khi nộp

Điền bìa và danh sách thành viên. Xác nhận yêu cầu định dạng từ thầy. Thực hiện review chéo, đẩy repo và chạy GitHub Actions. Ghi biên bản họp với ngày, người tham dự và quyết định thật. Đo lại trên máy chuẩn nếu cần đạt gate CV. Tự kiểm tra trích dẫn và tương đồng theo hệ thống của trường. Không có tuyên bố đã kiểm tra đạo văn bằng công cụ của trường trong bộ bàn giao này.

C3 Tái kiểm tra đóng gói

Giải nén vào thư mục mới, chạy demo, mở báo cáo Word/PDF và slide, kiểm tra số lượng CSV/hình và đọc metadata. Đối chiếu SHA256 modules với metadata trước khi dùng số liệu. Không yêu cầu Internet để chạy CLI sau khi môi trường đã cài đặt. Khi mất mạng ở lớp, dùng log console và CSV đã đóng gói làm phương án dự phòng.